



System Design Tutorial

Phase 4 — Ace the System Design Interview

A universal framework, 40 practice questions, and the soft skills that win offers

Volume 4 of 4 | The Framework · 10 Beginner · 15 Intermediate · 15 Advanced · Mistakes · Communication

Table of Contents

How the Interview Actually Works 6

The Universal Framework 7

Step 1 — Clarify requirements (5 min)	7
Step 2 — Estimate (3-5 min)	7
Step 3 — Define the API and data model (3-5 min)	7
Step 4 — High-level design (8-10 min)	7
Step 5 — Deep dive (10-15 min)	7
Step 6 — Identify bottlenecks & scale (5-8 min)	7
Step 7 — Wrap up (2-3 min)	8

Beginner Questions (1-10) 9

B1. What happens when you type a URL and press Enter?	9
B2. Horizontal vs vertical scaling — what's the difference?	9
B3. What is a load balancer and why use one?	9
B4. SQL vs NoSQL — how do you choose?	9
B5. What is caching, and what are the main strategies?	10
B6. What is a CDN and when would you use it?	10
B7. How do you make a web service highly available?	10
B8. Design a simple URL shortener.	10
B9. What's the difference between latency and throughput?	10
B10. What is a database index and why does it speed up queries?	11

Intermediate Questions (11-25) 12

11. Design a distributed unique ID generator.	12
12. Design a typeahead / autocomplete service.	12
13. Design a notification system (push, email, SMS).	12
14. Design a distributed cache.	12
15. How do you scale a relational database?	13
16. Design a web crawler.	13
17. Design a leaderboard / ranking system.	13

18. Design a distributed key-value store (like DynamoDB).	13
19. Design a rate limiter for a public API.	14
20. Design Pastebin.	14
21. Design a logging / metrics pipeline.	14
22. Design an online presence (online/last-seen) system.	14
23. Explain the CAP theorem with concrete examples.	14
24. Design a news feed.	14
25. Design a chat / messaging service.	15

Advanced Questions (26-40) 16

26. Design YouTube / Netflix	16
27. Design Uber / Lyft	16
28. Design WhatsApp / Messenger at scale	16
29. Design Google Drive / Dropbox	16
30. Design Twitter (full scale)	17
31. Design a distributed message queue (like Kafka)	17
32. Design a search engine (like Google Search)	17
33. Design a payment system (like Stripe)	17
34. Design Google Maps	18
35. Design a distributed file system (like GFS / HDFS)	18
36. Design a ticket-booking system (like Ticketmaster)	18
37. Design an ad click aggregation / analytics system	18
38. Design a recommendation system	19
39. Design a distributed lock / coordination service (like ZooKeeper / etcd)	19
40. Design a stock exchange / trading system	19

Common Mistakes — and How to Avoid Them 20

Communicating Your Design Clearly 21

Make your thinking visible	21
Structure out loud	21
Drive, but stay collaborative	21
Use the whiteboard well	21
Handle uncertainty gracefully	21

You're Ready — Final Words

22

How the Interview Actually Works

You now know the components, the principles, and how to design real systems. This final volume turns that knowledge into interview performance — because how you think out loud matters as much as what you know.

A system design interview is a 35–50 minute open-ended conversation. The interviewer gives you a deliberately vague prompt ("design Twitter") and watches *how you think*: do you ask good questions, structure your approach, make sound trade-offs, and communicate clearly? They are not looking for the one 'correct' architecture — there isn't one. They are evaluating your engineering judgement and your ability to collaborate.

This means a candidate who knows slightly less but communicates a clear, well-reasoned design will usually beat one who knows more but rambles, jumps around, or designs in silence. The framework on the next page is your scaffold: it keeps you structured, ensures you cover what matters, and signals to the interviewer that you've done this before.

What interviewers are really scoring

- **Structure**: do you drive the conversation methodically, or flail?
- **Trade-offs**: can you justify choices and name what you're giving up?
- **Scale awareness**: do you spot bottlenecks and know how to scale past them?
- **Communication**: are you clear, collaborative, and responsive to hints?
- **Depth**: can you go deep on at least one component when asked?

The Universal Framework

Memorise these seven steps. They work for any system design prompt and keep you on the rails when nerves hit.

- | | |
|----------------|---|
| 1. CLARIFY | -> requirements, scope, constraints (don't skip!) |
| 2. ESTIMATE | -> traffic, storage, bandwidth (back-of-envelope) |
| 3. API + MODEL | -> key endpoints and the data model |
| 4. HIGH-LEVEL | -> draw the boxes and arrows (the core design) |
| 5. DEEP DIVE | -> solve the 1-2 genuinely hard parts in detail |
| 6. SCALE IT | -> find bottlenecks; add cache/shard/replicate/LB |
| 7. WRAP UP | -> trade-offs, failure modes, what you'd improve |

Figure F.1 - The seven-step framework. Spend the most time on steps 4-6.

Step 1 — Clarify requirements (5 min)

Never start drawing immediately — the prompt is intentionally vague, and clarifying is the first thing interviewers grade. Separate **functional** requirements (what the system does) from **non-functional** ones (scale, latency, availability, consistency). Agree on scope explicitly: "I'll focus on posting and the feed, and treat auth as out of scope — is that OK?" This prevents you from designing the wrong thing.

Step 2 — Estimate (3-5 min)

Do quick back-of-the-envelope maths for requests/sec, storage, and bandwidth. State assumptions aloud ("assume 100M DAU"). The goal isn't precision — it's to reveal whether you're dealing with thousands or millions of operations per second, because that completely changes the design.

Step 3 — Define the API and data model (3-5 min)

List the key API endpoints (e.g. `createPost(userId, content)`) and sketch the main entities and how they relate. This forces concreteness and naturally exposes the data access patterns that drive your storage choices.

Step 4 — High-level design (8-10 min)

Draw the major components — clients, load balancer, services, databases, caches, queues, CDN — and the data flow between them. Walk the interviewer through a request end to end. Keep it simple first; you'll add sophistication when you scale.

Step 5 — Deep dive (10-15 min)

The interviewer will steer you into the hard part (or you should propose it): the feed fan-out, the geo-index, the transcoding pipeline. This is where most of your score is earned — show real depth on one or two components rather than shallow coverage of everything.

Step 6 — Identify bottlenecks & scale (5-8 min)

Proactively find the weak points: "the database will be the bottleneck at this read volume." Then apply the toolkit from Phase 2 — add caching, read replicas, sharding, more load balancing, async queues — and remove single points of failure for availability.

Step 7 — Wrap up (2-3 min)

Summarise the design, restate the key trade-offs you made, mention how the system handles failures, and note what you'd improve with more time. A crisp summary leaves a strong final impression.

Beginner Questions (1-10)

Foundational questions with full answers. Master these conceptual building blocks first — every harder question assumes them.

B1. What happens when you type a URL and press Enter?

This classic question tests whether you understand the whole request lifecycle. Walk through it:

- **DNS resolution:** the browser turns the domain into an IP address (checking caches first, then querying a DNS resolver).
- **Connection:** the browser opens a TCP connection to that IP and, for HTTPS, performs a TLS handshake to set up encryption.
- **Request routing:** the request often hits a CDN edge, then a load balancer / reverse proxy, which forwards it to an application server.
- **Server processing:** the app server runs logic, checks a cache, queries the database, and builds a response.
- **Response & render:** HTML returns; the browser parses it and fetches further assets (CSS/JS/images, usually from a CDN), then renders the page. Caching happens at many of these layers.

B2. Horizontal vs vertical scaling — what's the difference?

Vertical scaling (scale up) means making one machine more powerful (more CPU/RAM). It's simple and needs no code changes, but hits a hardware ceiling, gets expensive, and leaves a single point of failure. **Horizontal scaling (scale out)** means adding more machines behind a load balancer. It scales almost without limit and tolerates node failures, but requires the app to be designed for distribution — ideally **stateless** servers with shared state in a database or cache. Rule of thumb: scale up first for simplicity, scale out when you need limitless growth or high availability.

B3. What is a load balancer and why use one?

A load balancer sits in front of multiple servers and distributes incoming requests across them, presenting one address to the world. It enables horizontal scaling and high availability: it runs **health checks** and routes traffic away from failed servers automatically. **Layer 4** balancers route by IP/port (fast, content-blind); **Layer 7** balancers understand HTTP and can route by URL or cookie and terminate TLS. Common algorithms: round robin, least connections, and consistent hashing (to keep a client pinned to a server).

B4. SQL vs NoSQL — how do you choose?

Choose **SQL** when data is structured with clear relationships and you need **ACID** transactions and strong consistency — money, orders, inventory — or complex queries and joins. Choose **NoSQL** when you need massive horizontal scale, very high write throughput, a flexible/evolving schema, or a data shape that fits a specific model (key-value, document,

wide-column, graph), and you can accept **eventual consistency**. In practice, large systems mix both (polyglot persistence): SQL for the critical transactional core, NoSQL for scale-out pieces.

B5. What is caching, and what are the main strategies?

A cache is a fast (usually in-memory) store holding copies of frequently-used data so you avoid slow recomputation or database hits. Strategies: **cache-aside** (app checks cache, falls back to DB on a miss, then populates — the common default); **write-through** (write to cache and DB together, always fresh); **write-back** (write to cache, flush to DB later — fast but risks loss). When full, a cache **evicts** using a policy like **LRU** (least recently used). Caching shines when reads vastly outnumber writes and slightly stale data is acceptable. The hard part is **invalidation** — keeping cache and source in sync.

B6. What is a CDN and when would you use it?

A CDN is a global network of edge servers that cache your content close to users. When someone requests a file, they get it from a nearby edge instead of your distant origin, cutting latency and offloading your servers. Use a CDN for static assets (images, video, CSS, JS), for a globally distributed audience, for streaming, and to absorb traffic spikes. It's ideal for content that's the same for many users; highly personalised or rapidly-changing data needs short TTLs or different handling.

B7. How do you make a web service highly available?

Eliminate single points of failure by having more than one of everything critical. Run multiple application servers behind a load balancer with health checks; replicate the database with automatic failover to a standby; spread across multiple availability zones (and regions for the highest tiers). Add monitoring to detect failures fast, and design for **graceful degradation** — drop a non-essential feature rather than failing the whole page. Availability is measured in 'nines', and each extra nine costs roughly ten times more, so match the target to the cost of downtime.

B8. Design a simple URL shortener.

Map a generated **short key** to the long URL in a key-value store; redirect on lookup. Generate the key by encoding an incrementing counter in **Base62** (guaranteed unique, ~3.5 trillion 7-character keys) or by handing out pre-generated keys from a key-generation service. It's a **read-heavy** system, so put a cache (Redis) in front of the database to serve popular links. Use a 301 redirect to minimise server load, or a 302 if you need to track every click. (Full walkthrough in Phase 3.)

B9. What's the difference between latency and throughput?

Latency is how long a single request takes (the delay you feel); **throughput** is how many requests the system handles per second (total capacity). They're different dimensions and can trade off — batching raises throughput but adds latency. Optimise latency for interactive user-facing paths, throughput for bulk/background work. Always measure latency with **percentiles** (p99), never averages, because the slow tail is what frustrates real users.

B10. What is a database index and why does it speed up queries?

An index is a separate sorted data structure (commonly a **B-tree**) that lets the database find rows by a column's value without scanning the whole table — turning a slow linear scan into a fast logarithmic lookup. The trade-off: indexes consume storage and slow down writes (every write must update the indexes too), so you index the columns you frequently search, filter, or join on — not every column.

Intermediate Questions (11-25)

Fuller design problems with complete answers. These are the bread-and-butter of most interviews — practise explaining each one out loud in 5-7 minutes.

11. Design a distributed unique ID generator.

You need unique, roughly time-ordered 64-bit IDs across many servers, fast, with no central bottleneck. A plain database auto-increment is a single bottleneck; random UUIDs are unique but large and not sortable. The standard answer is **Twitter Snowflake**: build each ID from a timestamp + a machine ID + a per-machine sequence number.

```
SNOWFLAKE 64-bit ID  
[ 1 bit unused | 41 bits timestamp | 10 bits machine | 12 bits seq ]  
timestamp -> time-sortable      machine id -> no coordination needed  
sequence   -> many IDs per millisecond per machine
```

Figure 11.1 - A Snowflake ID packs time, machine, and a counter into 64 bits.

Each machine generates IDs locally with no coordination, they sort by creation time (great for databases), and the scheme yields millions of unique IDs per second. Watch out for clock skew between machines.

12. Design a typeahead / autocomplete service.

As a user types, return the top few completions almost instantly (within tens of milliseconds). Store known queries in a **trie** (prefix tree); at each node, precompute and cache the **top-k** most popular completions for that prefix so a lookup is a single fast read rather than a tree traversal. Rank by popularity/frequency, updated periodically from query logs (offline) rather than on every keystroke. Cache hot prefixes in memory, debounce client requests, and shard the trie by prefix for scale. Eventual consistency is fine — suggestions needn't be perfectly current.

13. Design a notification system (push, email, SMS).

Send notifications across multiple channels reliably at scale. A **notification service** accepts requests and places them on a **message queue**; per-channel **workers** consume the queue and call the right provider (APNs/FCM for push, an email provider, an SMS gateway). The queue decouples senders from delivery and absorbs spikes.

- **Reliability**: retries with backoff for failed sends; a dead-letter queue for repeated failures.
- **Idempotency**: dedupe so a retried request doesn't send the same notification twice.
- **User preferences & rate limiting**: respect opt-outs, quiet hours, and per-user caps.
- **Templating & priority**: separate high-priority (OTP) from bulk (marketing) traffic.

14. Design a distributed cache.

Build a cache that spans many nodes and survives failures. Spread keys across nodes using **consistent hashing** so adding/removing a node moves only a small fraction of keys.

Replicate each key to one or more other nodes for availability. Use an eviction policy (LRU) and TTLs, and a **cache-aside** pattern on the client side. Discuss handling **hot keys** (a single wildly popular key overloading one node — mitigate with local caching or key replication) and the **thundering herd** when a popular key expires (use locks or staggered TTLs). This is essentially how Redis Cluster and Memcached fleets operate.

15. How do you scale a relational database?

Apply these in order, escalating only as needed:

- **Indexing & query tuning**: fix slow queries first — the cheapest win.
- **Caching**: put Redis/Memcached in front to absorb repeated reads.
- **Read replicas**: replicate the primary and send reads to replicas (handles read-heavy loads; beware replication lag).
- **Vertical scaling**: a bigger box buys time.
- **Sharding**: split data across databases by a shard key when one machine can't hold the data or write load — powerful but complex (cross-shard queries and rebalancing get hard). Last resort.

16. Design a web crawler.

Fetch and process billions of web pages. A **URL frontier** (a prioritised queue) feeds a fleet of **fetcher workers** that download pages, extract links, and add new URLs back to the frontier. Store fetched content for indexing.

- **Politeness**: respect robots.txt and rate-limit per domain so you don't hammer any site.
- **Deduplication**: avoid re-crawling the same URL/content — a **Bloom filter** gives a memory-efficient 'have I seen this?' check.
- **Distributed & fault-tolerant**: partition the frontier across machines; checkpoint progress.
- **Freshness**: re-crawl important pages more often; avoid spider traps and infinite URL spaces.

17. Design a leaderboard / ranking system.

Show top-N players and a user's rank in real time. A **Redis sorted set** is the perfect fit: it keeps members ordered by score, with $O(\log N)$ updates and instant top-N and rank queries. For huge player counts, shard leaderboards (per region/league) or keep only the top scores precisely while approximating distant ranks. Persist the source of truth in a database; treat the sorted set as a fast, rebuildable view.

18. Design a distributed key-value store (like DynamoDB).

A highly available store that scales horizontally. **Partition** data across nodes with consistent hashing; **replicate** each key to N nodes. For consistency, use **quorums**: with N replicas, require W nodes to acknowledge a write and R nodes to serve a read; if $W + R > N$ you get strong consistency, while smaller values favour availability/latency (tunable consistency). Resolve conflicting concurrent writes with version vectors or last-write-wins. This is the

Amazon Dynamo design — an **AP** system that stays writable during partitions.

19. Design a rate limiter for a public API.

Cap each client to N requests per window and return 429 when exceeded, enforced at the API gateway. Because the gateway runs on many instances, counters must be **shared** — store them in **Redis** and update with **atomic** operations (INCR/EXPIRE or a Lua script) to avoid race conditions. Use **token bucket** to allow bursts or **sliding window** for smoothness (fixed windows allow a double-rate burst at the boundary). Decide **fail-open vs fail-closed** if Redis is down — usually fail-open for rate limiting. (Full walkthrough in Phase 3.)

20. Design Pastebin.

Store and share text snippets via a unique URL. Generate a short key (Base62, like the URL shortener) mapping to the paste. Store small pastes in a database; put large blobs in **object storage** (S3) and keep only metadata + a pointer in the DB. Support **expiration** via a TTL. It's read-heavy, so cache popular pastes and serve through a CDN. The data model and key generation mirror the URL shortener — a good example of reusing a pattern.

21. Design a logging / metrics pipeline.

Collect, process, and query huge volumes of logs and metrics. Lightweight **agents** on each server ship events to a high-throughput buffer (**Kafka**), which decouples producers from consumers and absorbs spikes. Stream processors aggregate and enrich events, then write to appropriate stores: a **time-series database** for metrics, and a search store (e.g. Elasticsearch) for log search. Dashboards and alerts read from there. Use **sampling** and retention/rollups to control cost; this is a throughput-oriented system where some loss is tolerable.

22. Design an online presence (online/last-seen) system.

Show whether users are online. Clients send periodic **heartbeats**; the presence service marks a user online and records a last-seen timestamp in a fast store (Redis). If no heartbeat arrives within a timeout, mark them offline. At scale, the challenge is fan-out of status changes to interested friends — publish presence changes via pub/sub rather than having everyone poll. Tolerate slight staleness (eventual consistency) to keep the system cheap and scalable.

23. Explain the CAP theorem with concrete examples.

In a distributed store, during a network **partition** you can guarantee either **Consistency** or **Availability**, not both (partition tolerance is mandatory because networks do fail). A **CP** system (e.g. a banking ledger, or etcd/ZooKeeper) refuses or blocks requests during a partition to avoid returning wrong data — correct but temporarily unavailable. An **AP** system (e.g. Cassandra, DynamoDB, a social 'like' count) keeps responding with possibly-stale data — available but eventually consistent. The right choice depends on whether stale data or unavailability is worse for your use case. PACELC adds that even without partitions you trade latency against consistency.

24. Design a news feed.

Show each user recent posts from people they follow. The central decision is **fan-out on write** (precompute each follower's feed when someone posts — instant reads, but a write explosion for celebrities) vs **fan-out on read** (assemble the feed at read time — cheap writes, slow reads). Real systems use a **hybrid**: push for normal users, pull for celebrities, merged at read time. Store timelines in Redis, media in object storage + CDN, and use a queue for fan-out. (Full walkthrough in Phase 3.)

25. Design a chat / messaging service.

Deliver messages in real time, in order, with no loss. Clients hold persistent **WebSocket** connections to **connection servers**; a **session registry** (Redis) maps each user to their connection server so messages can be routed for instant delivery. Persist messages in a wide-column store (Cassandra) keyed by conversation. Use **acknowledgements** for sent/delivered/read status and to guarantee delivery; sequence numbers for ordering; and **push notifications** for offline users. (Full walkthrough in Phase 3.)

Advanced Questions (26-40)

Big, open-ended designs. Here you get hints and an approach framework rather than full answers — the goal is to know where the hard problems are and how to attack them. Apply the seven-step framework to each.

26. Design YouTube / Netflix

The core challenge: storing and streaming enormous volumes of video to a global audience with no buffering.

- **Key components:** upload service, object storage for video, an async transcoding pipeline (queue + workers), metadata DB, a recommendation service, and — above all — a CDN.
- **Deep-dive on:** transcoding each video into a quality ladder split into chunks, adaptive bitrate streaming (HLS/DASH), and CDN strategy (e.g. Netflix Open Connect inside ISPs).
- **Watch out for:** storage cost from many resolutions, pre-transcode vs on-demand, and approximate view counts at scale.

27. Design Uber / Lyft

The core challenge: matching riders to nearby drivers in real time while millions of cars stream GPS every few seconds.

- **Key components:** a high-write location ingestion service, an in-memory geo-index (Redis), a matching service, a trip service (consistent store), pricing/surge, and real-time notifications.
- **Deep-dive on:** geospatial indexing (geohashing / QuadTrees / S2) to find nearby drivers fast, and handling the GPS write storm in memory rather than on disk.
- **Watch out for:** update-frequency vs load, and splitting consistency — locations best-effort (AP), trip/payment strongly consistent (CP).

28. Design WhatsApp / Messenger at scale

The core challenge: real-time, reliable, ordered delivery across hundreds of millions of simultaneous connections.

- **Key components:** WebSocket connection servers, a session/presence registry, a chat service, a wide-column message store, and a push-notification service for offline users.
- **Deep-dive on:** routing a message to the recipient's connection server, acknowledgements for delivery guarantees, ordering via sequence numbers, and group fan-out.
- **Watch out for:** managing millions of stateful connections, end-to-end encryption implications, and presence fan-out.

29. Design Google Drive / Dropbox

The core challenge: syncing files reliably across a user's devices with efficient storage and conflict handling.

- **Key components:** a metadata service (file tree, versions, permissions), object storage for file content, a sync service, and a notification channel to push changes to devices.
- **Deep-dive on:** splitting files into **chunks** so only changed chunks upload/download, **deduplication** of identical chunks, and conflict resolution when two devices edit offline.
- **Watch out for:** large-file uploads (resumable, chunked), sync efficiency, and keeping metadata consistent.

30. Design Twitter (full scale)

The core challenge: the feed fan-out problem plus tweets, follows, search, and trending at massive read scale.

- **Key components:** tweet service, social graph service, fan-out service + timeline cache, search service, media via object storage + CDN, and a trending/analytics pipeline.
- **Deep-dive on:** hybrid fan-out (push for most, pull for celebrities), and building trending topics from a real-time stream (Kafka + stream processing).
- **Watch out for:** the celebrity hot-key problem, timeline storage cost, and ranking vs chronological feeds.

31. Design a distributed message queue (like Kafka)

The core challenge: a durable, high-throughput, ordered log that many producers and consumers can share and replay.

- **Key components:** topics split into **partitions**, an append-only commit log per partition, replication across brokers, and consumer groups tracking **offsets**.
- **Deep-dive on:** how partitions provide parallelism and ordering (ordered within a partition, not across), replication for durability, and consumer offset management for replay and at-least-once delivery.
- **Watch out for:** ordering guarantees, rebalancing consumers, and exactly-once vs at-least-once semantics.

32. Design a search engine (like Google Search)

The core challenge: crawling the web, indexing it, and returning relevant ranked results in milliseconds.

- **Key components:** a distributed crawler, an **inverted index** (word → list of documents), a ranking service, and a low-latency query-serving tier.
- **Deep-dive on:** building and sharding the inverted index, and ranking (relevance signals, link analysis like PageRank), with heavy caching of popular queries.
- **Watch out for:** index size and freshness, query latency at scale, and spam/quality filtering.

33. Design a payment system (like Stripe)

The core challenge: moving money correctly — never losing or double-charging — while integrating external banks.

- **Key components:** a payment service, a **double-entry ledger** as the source of truth, idempotency keys, integrations with payment gateways/banks, and a reconciliation process.
- **Deep-dive on: idempotency** (a retried request must not charge twice), strong consistency for balances, and handling asynchronous, sometimes-slow external bank responses.
- **Watch out for:** consistency above all (this is firmly CP), audit/compliance, and partial failures mid-transaction.

34. Design Google Maps

The core challenge: storing the world's map, computing fast routes, and estimating arrival times at global scale.

- **Key components:** map tile storage + CDN, a routing service over a road **graph**, a geocoding service, and a real-time traffic pipeline feeding ETAs.
- **Deep-dive on:** shortest-path routing on a huge graph (Dijkstra/A*, precomputed contraction hierarchies for speed) and serving map tiles efficiently.
- **Watch out for:** graph size and partitioning, incorporating live traffic, and balancing precomputation vs query-time cost.

35. Design a distributed file system (like GFS / HDFS)

The core challenge: storing massive files reliably across many commodity machines that routinely fail.

- **Key components:** a **master/namenode** holding metadata (file → chunk locations) and many **chunkservers/datanodes** storing large fixed-size chunks, each replicated 3×.
- **Deep-dive on:** how the master tracks chunks and orchestrates replication, and how reads/writes flow directly to chunkservers to keep the master out of the data path.
- **Watch out for:** the master as a potential bottleneck/SPOF (needs HA), and re-replicating chunks when a node dies.

36. Design a ticket-booking system (like Ticketmaster)

The core challenge: selling limited inventory under extreme contention without double-booking a seat.

- **Key components:** an inventory/seat service with strong consistency, a temporary **seat-hold/locking** mechanism with expiry, a payment step, and a queue to throttle massive spikes.
- **Deep-dive on:** preventing double-booking via locks or atomic conditional updates/transactions, and holding seats during checkout then releasing them on timeout.
- **Watch out for:** huge flash-sale spikes (virtual waiting room), consistency over availability for inventory, and fairness.

37. Design an ad click aggregation / analytics system

The core challenge: counting billions of ad events in near-real-time, accurately, despite duplicates and late data.

- **Key components:** event ingestion (Kafka), stream-processing aggregators, a fast store for live counts, and a batch layer for accurate reprocessing.
- **Deep-dive on:** deduplication and (near) exactly-once counting, windowed aggregation, and handling late/out-of-order events.
- **Watch out for:** the trade-off between real-time approximate counts and batch-accurate corrections (a lambda-style architecture), and fraud filtering.

38. Design a recommendation system

The core challenge: suggesting relevant items to users from a catalogue of millions, fast and personalised.

- **Key components:** a **candidate generation** stage (narrow millions to hundreds) plus a **ranking** stage (ML model scores the candidates), a feature store, and offline training pipelines.
- **Deep-dive on:** the two-stage retrieve-then-rank pattern, collaborative vs content-based filtering, and serving precomputed candidates with low latency.
- **Watch out for:** the cold-start problem (new users/items), feedback loops, and balancing offline computation with online freshness.

39. Design a distributed lock / coordination service (like ZooKeeper / etcd)

The core challenge: letting many machines agree on shared state (locks, leader election, config) despite failures.

- **Key components:** a small cluster of nodes running a **consensus** algorithm (Raft or Paxos) over a replicated log, exposing a consistent key-value/namespace API.
- **Deep-dive on:** how consensus achieves agreement and survives a minority of node failures, plus leader election and lock leases with timeouts.
- **Watch out for:** this is strongly consistent (CP); avoid using it as a general database, and handle client session/lease expiry carefully.

40. Design a stock exchange / trading system

The core challenge: matching buy and sell orders fairly with ultra-low, predictable latency and perfect correctness.

- **Key components:** an order gateway, an in-memory **matching engine** maintaining the **order book** per symbol, a durable event log of all orders, and market-data publishers.
- **Deep-dive on:** the matching engine (price-time priority on an in-memory order book) and durability via an append-only log so state can be rebuilt after a crash.
- **Watch out for:** fairness and determinism, microsecond-level latency (single-threaded per symbol, no GC pauses), and strict auditability.

Common Mistakes — and How to Avoid Them

Most candidates don't fail because they lack knowledge — they fail on avoidable process mistakes. Here are the big ones.

- **Jumping straight into a solution.** Diving into boxes before clarifying requirements is the number-one mistake. *Fix:* always spend the first few minutes on requirements and scope.
- **Skipping estimation.** Without rough numbers you can't justify any decision. *Fix:* do quick back-of-envelope maths and state assumptions aloud.
- **Designing in silence.** The interviewer can't score thoughts they can't hear. *Fix:* narrate your reasoning continuously, especially the *why*.
- **Over-engineering.** Proposing Kafka, microservices, and five databases for a simple problem signals poor judgement. *Fix:* start simple, add complexity only when scale demands it.
- **Never mentioning trade-offs.** Presenting one design as obviously correct looks naive. *Fix:* for every major choice, say what you're giving up and why the trade is right here.
- **Ignoring the bottleneck.** Not identifying where the system strains. *Fix:* proactively call out the bottleneck ("the DB at this read volume") and address it.
- **Forgetting failures.** Designing only the happy path. *Fix:* ask 'what if this component dies?' and add redundancy/failover.
- **Going too broad, never deep.** Touching everything shallowly. *Fix:* cover the high level, then go genuinely deep on one or two hard parts.
- **Ignoring the interviewer's hints.** Hints are gifts steering you to what they want to see. *Fix:* treat any nudge or question as a signal and follow it.
- **Getting stuck on perfection.** Freezing because you can't find the 'best' answer. *Fix:* make a reasonable choice, state its trade-off, and move on — you can revisit.

Communicating Your Design Clearly

The interview is a collaboration, not an exam. How you communicate often decides the outcome.

Make your thinking visible

Narrate as you go: "I'll use a cache here *because* reads dominate writes." The reasoning matters more than the conclusion — it shows the judgement they're hiring for. When you make a choice, immediately give the one-line justification and the trade-off.

Structure out loud

Signal where you are in the process: "First let me clarify requirements, then estimate, then sketch the high-level design." This makes you easy to follow and demonstrates a methodical approach. Use the seven-step framework as your visible scaffold.

Drive, but stay collaborative

Lead the conversation, but treat it as a dialogue. Check in ("Does it make sense to go deeper on the feed generation next?"), invite input, and adapt when the interviewer redirects. Confident leadership plus responsiveness is the sweet spot — not domineering, not passive.

Use the whiteboard well

Draw clear boxes and labelled arrows; keep the diagram tidy and evolve it as you scale. A clean visual lets you and the interviewer reason about the system together. Point to components as you discuss them.

Handle uncertainty gracefully

If you don't know something, say so honestly and reason from fundamentals rather than bluffing (interviewers spot bluffing instantly). "I haven't used that specific tool, but the principle is ..." is a strong answer. Stay calm, think aloud, and treat hints as help, not criticism.

The one-minute pre-flight checklist

- Did I **clarify** requirements and scope before designing?
- Did I **estimate** scale and state my assumptions?
- Did I explain the **why** behind each major choice?
- Did I name the **trade-offs** and the **bottleneck**?
- Did I go **deep** on at least one component and consider **failures**?

You're Ready — Final Words

Four volumes ago you started from zero. Look how far you've come.

You now understand the components that systems are built from, the principles that hold them together, how to design six real systems end to end, and how to perform under interview pressure. That is a complete foundation — the same foundation working engineers use every day.

The last ingredient is **practice**. Read a design here, then close the book and rebuild it on a blank page, out loud, as if an interviewer were watching. Do the advanced questions properly — spend 40 minutes each applying the framework. Practise with a friend or out loud to yourself. The method becomes second nature only through repetition, and once it does, even an unfamiliar prompt becomes just another walk through the seven steps.

Your complete journey

- **Phase 1:** the core components — the vocabulary of systems.
- **Phase 2:** the principles — scalability, availability, consistency, and trade-offs.
- **Phase 3:** six real designs — the method applied end to end.
- **Phase 4:** the framework, 40 questions, and the communication skills to deliver under pressure.
- Now go and practise — you have everything you need. Good luck.